

# Gabarito

|             |                             |
|-------------|-----------------------------|
| 🕒 Criado em | 9 de setembro de 2026 17:27 |
| ☰ Tags      |                             |

## 1. Com base na figura 1 explique Sobrecarga e Sobreposição (Overriding).

**Sobrecarga (Overloading)** acontece quando uma classe possui métodos com o **mesmo nome**, mas com **parâmetros diferentes**. A ideia é permitir que uma mesma operação possa ser utilizada de maneiras diferentes dependendo dos argumentos fornecidos.

Exemplo:

```
autonomia()  
autonomia(int distancia)  
autonomia(int distancia, int passageiros)
```

O nome do método é o mesmo, mas a quantidade ou o tipo dos parâmetros muda.

**Sobreposição (Overriding)** acontece quando uma **subclasse redefine um método que já existe na superclasse**, fornecendo uma implementação específica para aquele tipo de objeto.

Por exemplo, se `Carro` possui um método `Consumo()`, `CarroAlcool` pode sobrescrever esse método para calcular o consumo de uma maneira específica para carros a álcool.

**Diferença principal:**

- Sobrecarga → mesmo método, **parâmetros diferentes**.
- Sobreposição → método herdado, **comportamento redefinido pela subclasse**.

## 2. Com base na figura 1 marque verdadeiro ou falso para as linhas da

tabela abaixo. Para as alternativas falsas explique a sua marcação.

1. `Carro c = new CarroAlcool();`

**VERDADEIRO.**

Aqui é importante entender que existem **dois tipos envolvidos**:

```
Carro c = new CarroAlcool();
```

- `Carro` é o **tipo da variável/referência**.
- `CarroAlcool` é o **tipo do objeto que foi criado** pelo `new`.

Isso é possível porque `CarroAlcool` é uma classe derivada de `Carro`, ou seja, **CarroAlcool é um tipo de Carro**.

É como dizer:

“Tenho uma referência do tipo mais geral `Carro`, apontando para um objeto mais específico, que é `CarroAlcool`.”

Essa forma de declaração é muito importante para o **polimorfismo**, porque permite trabalhar com diferentes tipos de objetos por meio de uma referência comum.

2. `CarroFlex c1 = new CarroGasolina();`

**FALSO.**

O problema é que `CarroFlex` e `CarroGasolina` são **tipos diferentes de especialização**.

Para essa atribuição funcionar, `CarroGasolina` precisaria ser um tipo de `CarroFlex`, ou seja, precisaria existir uma relação de herança compatível entre eles.

Não basta as duas classes serem relacionadas à classe `Carro`.

Por exemplo:

```
Carro
 /  \
```

```
CarroFlex CarroGasolina
```

Nesse caso, `CarroFlex` e `CarroGasolina` são classes derivadas de `Carro`, mas **uma não é filha da outra**.

Portanto:

```
CarroFlex c1 = new CarroGasolina();
```

não é permitido.

Uma forma simples de lembrar:

Classes “irmãs” na herança não podem ser atribuídas diretamente uma à outra.

### 3. `Carro c2 = new Carro();`

**FALSO**, considerando que `Carro` é uma **classe abstrata** na Figura 1.

Uma classe abstrata pode ser utilizada como **tipo de referência**, mas não podemos criar diretamente um objeto dela.

Por exemplo, isto pode existir:

```
Carro c;
```

Mas isto não:

```
Carro c = new Carro();
```

O motivo é que a classe abstrata representa uma abstração que serve como base para outras classes. Ela pode definir características e comportamentos que serão utilizados pelas subclasses, mas não representa um objeto concreto que possa ser instanciado diretamente.

Por isso podemos ter:

```
Carro c = new CarroAlcool();
```

mas não:

```
Carro c = new Carro();
```

4. `CarroGasolina c3 = new CarroGasolina();`

**VERDADEIRO.**

Aqui o tipo da referência e o tipo do objeto são o mesmo:

```
CarroGasolina c3 = new CarroGasolina();
```

Estamos criando diretamente uma instância de `CarroGasolina` e armazenando essa referência em uma variável também do tipo `CarroGasolina`.

Como `CarroGasolina` é uma classe concreta, a instanciação é permitida.

5. `c.Consumo();`

**VERDADEIRO.**

A variável `c` foi declarada como:

```
Carro c = new CarroAlcool();
```

Portanto, a referência `c` é do tipo `Carro`.

Se `Consumo()` pertence à classe `Carro`, o método pode ser chamado através dessa referência.

O ponto importante aqui é separar:

**O tipo da referência determina quais métodos podem ser acessados pela variável.**

Ao mesmo tempo, se `CarroAlcool` tiver uma implementação sobrescrita de `Consumo()`, o comportamento específico de `CarroAlcool` poderá ser executado. É justamente aí que o **polimorfismo** aparece.

6. `c1.Consumo();`

**FALSO.**

Na questão 2, temos:

```
CarroFlex c1 = new CarroGasolina();
```

Essa própria declaração é **inválida**, porque `CarroGasolina` não é um `CarroFlex`. São especializações diferentes de `Carro`.

Portanto, a variável `c1` **não pode ser criada dessa forma**. Consequentemente, não podemos utilizar:

```
c1.Consumo();
```

Aqui é importante não pensar apenas:

“O método `Consumo()` existe?”

Primeiro precisamos perguntar:

“Essa referência pode existir com esse tipo e esse objeto?”

Como `CarroGasolina` não pode ser atribuído a uma referência `CarroFlex`, `c1` já está incorreto desde a declaração.

Isso é justamente uma aplicação da **compatibilidade entre tipos e da herança**: uma referência de uma classe pode apontar para um objeto de uma subclasse compatível, mas não para qualquer outra classe que esteja apenas no mesmo nível da hierarquia.

## 7. `c2.autonomia()`

**FALSO.**

O problema começa na própria declaração anterior:

```
Carro c2 = new Carro();
```

Como `Carro` é abstrata, essa criação de objeto não é permitida.

Então `c2` não pode ser criado dessa maneira.

Além disso, para saber se `autonomia()` poderia ser chamada, também precisamos observar se esse método está disponível para o tipo da referência `Carro`.

A ideia importante é:

Antes de analisar a chamada de um método, precisamos verificar se a referência foi criada corretamente e qual é o seu tipo.

8. `c3.autonomia(10,20);`

**FALSO.**

Na linha 4 temos:

```
CarroGasolina c3 = new CarroGasolina();
```

A referência `c3` é do tipo `CarroGasolina`.

Pela Figura 1, o método `autonomia()` disponível para essa referência recebe **apenas um parâmetro**. Portanto, podemos ter algo como:

```
c3.autonomia(10);
```

mas não:

```
c3.autonomia(10,20);
```

porque estamos passando **dois parâmetros** para um método cuja assinatura aceita apenas **um**.

Aqui entra diretamente o conceito de **sobrecarga**: para chamar `autonomia(10, 20)`, teria que existir uma outra versão do método `autonomia` que recebesse dois parâmetros. Se essa sobrecarga não existe na classe/referência, a chamada é inválida.

9. `((CarroAlcool)c).autonomia(10);`

**VERDADEIRO.**

Nós temos:

```
Carro c = new CarroAlcool();
```

A variável `c` tem tipo `Carro`, embora o objeto real seja `CarroAlcool`.

Então, por meio de `c`, só podemos acessar aquilo que está disponível para o tipo `Carro`.

O trecho:

```
((CarroAlcool)c)
```

faz uma **conversão explícita da referência**, dizendo:

“Eu sei que esse objeto é um `CarroAlcool`; trate essa referência como `CarroAlcool`.”

Depois disso podemos chamar:

```
.autonomia(10)
```

porque essa versão específica do método está disponível em `CarroAlcool`.

É importante perceber que o cast **não transforma o objeto em outro objeto**. O objeto continua sendo o mesmo `CarroAlcool`. O que muda é a forma como estamos tratando a referência naquele momento.

### 3. Explique a frase:

“Podemos criar variáveis do tipo de uma Classe Abstrata, porém não podemos criar instâncias de uma classe Abstrata”.

A frase significa que uma classe abstrata pode ser utilizada como **tipo de uma variável**, mas não pode ser utilizada diretamente para criar um objeto.

Podemos ter:

```
Carro c;
```

porque `Carro` pode ser um tipo de referência.

Também podemos ter:

```
Carro c = new CarroAlcool();
```

porque `CarroAlcool` é uma especialização de `Carro`.

Porém, não podemos fazer:

```
Carro c = new Carro();
```

se `Carro` for abstrata.

A ideia é que a classe abstrata funciona como uma **base conceitual** para suas subclasses. Ela define aquilo que os diferentes tipos de carro têm em comum, enquanto as classes concretas representam os objetos que realmente podem ser criados.

## 4. Encapsulamento, redução de acoplamento e complexidade

Imagine um sistema bancário.

Sem encapsulamento, outras partes do sistema poderiam alterar diretamente informações internas da conta:

```
conta.saldo = -500;
```

Isso cria problemas porque qualquer parte do sistema precisa conhecer e respeitar as regras internas da conta.

Com encapsulamento, a própria classe controla como seu estado pode ser alterado:

```
conta.sacar(500);
```

A classe `Conta` pode verificar internamente se existe saldo suficiente, registrar a operação e atualizar o saldo.

Isso reduz o **acoplamento**, porque as outras partes do sistema não precisam conhecer todos os detalhes de como o saldo é armazenado ou atualizado.

Também reduz a **complexidade**, porque cada parte precisa conhecer apenas aquilo que realmente precisa para utilizar a classe.

Então, o encapsulamento não existe simplesmente para colocar atributos como `private`.

Seu objetivo maior é:



esconder detalhes internos, controlar o acesso ao estado e diminuir a dependência entre as partes do sistema.

## 5. Principais características de Classe e Objeto

### Classe

A **classe** representa uma abstração ou definição de um determinado tipo de entidade no sistema. Ela descreve quais **características e comportamentos** os objetos daquele tipo terão.

Por exemplo, uma classe `Carro` pode definir:

- quais informações um carro possui;
- quais comportamentos ele possui;
- quais regras seus objetos devem seguir.

A classe, portanto, **define a estrutura e os comportamentos**, mas não representa, por si só, um objeto específico existente no sistema.

### Objeto

O **objeto** é uma instância concreta de uma classe.

Quando um objeto é criado, ele **herda as características e comportamentos definidos pela classe** e passa a possuir um **estado próprio**, que representa os valores específicos daquele objeto.

Além disso, o objeto possui um **ciclo de vida**: ele é criado, pode ser utilizado e, posteriormente, deixa de existir.

Outra característica importante é a **identidade**. Cada objeto é uma entidade individual dentro do sistema, mesmo que existam vários objetos da mesma classe.

Por exemplo, podemos ter:

Classe: Carro

Objetos:

```
Carro 1 → modelo = "Civic", cor = "preto"
Carro 2 → modelo = "Onix", cor = "branco"
```

Os dois objetos pertencem à mesma classe e possuem os comportamentos definidos por `Carro`, mas cada um possui **seu próprio estado, ciclo de vida e identidade**.

## 6. Polimorfismo — Polígono, Triângulo, Retângulo e Hexágono

Podemos ter uma classe mais geral:

```
abstract class Poligono {
    public abstract double calculoArea();
}
```

E classes específicas:

```
class Triangulo extends Poligono {
    public double calculoArea() {
        // cálculo da área do triângulo
    }
}

class Retangulo extends Poligono {
    public double calculoArea() {
        // cálculo da área do retângulo
    }
}

class Hexagono extends Poligono {
    public double calculoArea() {
        // cálculo da área do hexágono
    }
}
```

Todas possuem o mesmo método:

```
calculoArea()
```

Mas cada uma possui uma implementação adequada à sua própria forma.

Podemos então trabalhar com elas por meio do tipo mais geral:

```
Poligono p;
```

E esse `p` pode representar um:

```
Triângulo  
Retângulo  
Hexágono
```

O método chamado continua sendo:

```
calculoArea()
```

mas o comportamento executado depende do objeto concreto.

Esse é o **polimorfismo**: uma mesma interface/operação pode representar comportamentos diferentes dependendo do objeto.

## Ganho do polimorfismo

O principal ganho é permitir que o código trabalhe com uma **abstração comum**, sem precisar conhecer antecipadamente todos os tipos específicos.

Por exemplo, um sistema pode receber vários polígonos e simplesmente pedir:

```
poligono.calculoArea();
```

Sem precisar criar uma lógica enorme dizendo:

```
se for Triângulo → faça isso  
se for Retângulo → faça aquilo  
se for Hexágono → faça outra coisa
```

Isso facilita **extensão, manutenção e reutilização do código**.

## 7. Abstração e o surgimento da POO

A abstração está relacionada à necessidade de lidar com a **complexidade dos sistemas**.

Quando um sistema cresce, tentar trabalhar diretamente com todos os detalhes internos torna o desenvolvimento mais difícil.

A abstração permite representar apenas aquilo que é **relevante para determinado contexto**, escondendo detalhes que não precisam estar expostos.

A POO utiliza essa ideia para organizar o sistema em entidades que possuem responsabilidades próprias.

Por exemplo, em um sistema bancário podemos trabalhar com:

```
Conta
Cliente
Transferência
Cartão
```

Não precisamos representar cada detalhe físico ou interno dessas entidades. Criamos abstrações que representam aquilo que interessa ao sistema.

Assim, a abstração ajuda a:

- controlar a complexidade;
- organizar o sistema;
- separar responsabilidades;
- facilitar a compreensão;
- esconder detalhes desnecessários.

Por isso ela está diretamente relacionada à motivação da POO: **criar uma maneira mais organizada de representar e controlar a complexidade de sistemas de software**.

## 8. Programação Funcional x Programação Orientada a Objetos

A principal diferença está na **forma de organizar o pensamento e estruturar o programa**.

## Programação Funcional

Na programação funcional, o foco está principalmente em **funções e transformação de dados**.

A ideia é trabalhar com funções que recebem valores e produzem resultados, buscando reduzir efeitos colaterais e alterações de estado quando possível.

O programa é pensado mais como uma composição de operações.

## Programação Orientada a Objetos

Na POO, o foco está em **objetos que possuem estado e comportamento**, organizando o sistema em entidades e responsabilidades.

Um objeto pode possuir:

Estado → informações que ele possui  
Comportamento → operações que ele realiza

Além disso, a POO utiliza mecanismos como:

- abstração;
- encapsulamento;
- herança;
- polimorfismo.